



IES PAZO DA MERCÉ

*Ciclo Formativo de Grao Superior en Desenvolvemento
de Aplicacións Web.*

*Plataforma Web de Práctica Musical
Interactiva*

<i>Proxecto presentado por:</i>	<i>Alejandro Aballe Peña</i>
<i>Titor/a:</i>	<i>Hector Gómez</i>
<i>Data:</i>	<i>10 de Mayo de 2026</i>

Resumo

O presente proxecto ten como obxectivo o deseño e desenvolvemento dunha aplicación web orientada á práctica e xestión do estudo musical. A aplicación permitirá aos usuarios rexistrarse na plataforma, acceder a ferramentas interactivas e organizar e rexistrar o seu progreso de práctica de maneira estruturada.

No contexto actual de dixitalización do ensino e do uso crecente de ferramentas en liña, resulta de especial interese dispoñer de aplicacións web accesibles desde calquera dispositivo e localización. Este proxecto pretende integrar coñecementos adquiridos ao longo do ciclo formativo de Desenvolvemento de Aplicacións Web, tales como programación orientada a obxectos, desenvolvemento frontend con linguaxes de marcas e JavaScript, implementación de bases de datos relacionais e deseño dunha arquitectura cliente-servidor.

Durante o desenvolvemento aplicaranse metodoloxías de planificación, análise e deseño, así como a realización dun plan de probas para garantir o correcto funcionamento da aplicación. Ademais, incluírase a documentación técnica necesaria para a súa implantación, así como un plan empresarial que contextualice a posible integración da aplicación nun entorno profesional.

O resultado final será unha aplicación web funcional, escalable e documentada, que demostre de forma integrada as competencias adquiridas ao longo do ciclo formativo.

Palabras clave: Aplicación web, práctica musical, base de datos, cliente-servidor, JavaScript

Abstract

The aim of this project is the design and development of a web application focused on music practice management. The application will allow users to register, track users' practice progress, access interactive tools and organize their musical progress in a structured way.

In the current context of digital transformation and the growing use of online learning tools, web applications accessible from any device have become increasingly relevant. This project integrates the knowledge acquired throughout the Web Application Development vocational program, including object-oriented programming, frontend development using markup languages and JavaScript, relational database implementation, and client-server architecture design.

The development process includes planning, analysis, design, implementation and testing phases to ensure the correct functionality of the system. In addition, technical documentation and a business plan are provided to contextualize the application within a professional environment.

The final result is a functional, scalable and well-documented web application that demonstrates the integrated competencies acquired during the training program.

Keywords: Web application, music practice, relational database, client-server architecture, JavaScript

Índice

Sumario

Introdución.....	6
Contexto e xustificación.....	7
Obxectivos.....	9
Obxectivo xeral.....	9
Obxectivos específicos.....	9
Fases de traballo.....	10
Fase de estudo.....	10
Plan de empresa.....	10
Planificación.....	14
Análise e deseño.....	15
Desenvolvemento.....	21
Entorno de desenvolvemento e ferramenta.....	21
Política de copias de seguridade.....	22
Programación Orientada a Obxectos (POO).....	23
Plan de probas.....	26
Requisitos mínimos.....	29
Documentación.....	31
Documentación interna do código.....	31
Documentación da API REST.....	31

Diagrama de clases UML.....	.33
Conclusionés.....	.35
Propostas de mellora e valoración.....	.37
Bibliografía.....	.38

1. Introducción

O presente proxecto céntrase no deseño e desenvolvemento dunha aplicación web orientada á práctica e xestión do estudo musical. A elección desta temática xorde da crecente dixitalización dos procesos de aprendizaxe e da necesidade de integrar ferramentas tecnolóxicas que faciliten a organización e o seguimento da práctica musical nun único entorno accesible.

Na actualidade, o uso de plataformas dixitais está cada vez máis presente tanto no ámbito educativo como no profesional. No contexto das aplicacións web, as empresas e desenvolvedores buscan solucións que permitan ofrecer servizos accesibles desde distintos dispositivos e que faciliten a xestión de información e a interacción cos usuarios. No ámbito musical, existen múltiples recursos dixitais destinados á aprendizaxe e práctica de instrumentos; porén, moitos deles atópanse dispersos ou carecen dunha estrutura que permita ao usuario rexistrar, organizar e avaliar o seu progreso de forma sistemática. Detectouse, polo tanto, a oportunidade de desenvolver unha solución web que centralice estas funcionalidades.

O interese deste traballo radica na aplicación integrada dos coñecementos adquiridos ao longo do ciclo formativo de Desenvolvemento de Aplicacións Web. Preténdese demostrar a capacidade para analizar requisitos, deseñar unha arquitectura cliente-servidor, modelar unha base de datos relacional e implementar unha aplicación empregando linguaxes e tecnoloxías propias do desenvolvemento web.

Co desenvolvemento deste proxecto búscase obter como resultado unha aplicación web funcional, estruturada e documentada, que permita aos usuarios acceder a un catálogo de tablaturas, utilizar ferramentas de apoio á práctica musical e rexistrar o seu progreso. O enfoque adoptado basearase nun proceso estruturado que inclúe as fases de estudo, planificación, análise, deseño, desenvolvemento, probas e documentación, garantindo así a coherencia e calidade do produto final.

1.1. Contexto e xustificación

No contexto actual da transformación dixital, as aplicacións web convertéronse nunha das principais solucións tecnolóxicas empregadas por empresas e organizacións para ofrecer servizos accesibles desde diferentes dispositivos e localizacións. A crecente demanda de plataformas en liña que permitan xestionar contidos, interactuar cos usuarios e ofrecer ferramentas especializadas representa unha oportunidade de desenvolvemento dentro do sector tecnolóxico.

No ámbito musical, especialmente no relacionado coa aprendizaxe autodidacta e a práctica de instrumentos, existe unha ampla variedade de recursos dixitais como tablaturas, vídeos ou ferramentas de apoio ao estudo. Porén, moitas destas solucións atópanse distribuídas en diferentes plataformas, o que dificulta a organización do contido e o seguimento do progreso por parte dos usuarios. Isto xera a necesidade de desenvolver plataformas que integren estes recursos nun único entorno accesible e estruturado.

Desde o punto de vista empresarial, o desenvolvemento de aplicacións web especializadas representa unha oportunidade para ofrecer servizos dixitais adaptados a nichos concretos de usuarios, como poden ser os músicos ou estudantes de instrumentos. Este tipo de solucións permiten centralizar contido, facilitar a interacción entre usuarios e ofrecer ferramentas que melloren a experiencia de uso.

A elección deste proxecto responde tamén ao interese por aplicar de maneira práctica os coñecementos adquiridos ao longo do ciclo formativo de Desenvolvemento de Aplicacións Web. A realización desta aplicación permitirá traballar aspectos clave do desenvolvemento web, como o deseño de bases de datos relacionais, a implementación dunha arquitectura cliente-servidor, a xestión de usuarios e permisos, e o desenvolvemento de interfaces dinámicas mediante tecnoloxías web actuais.

Deste modo, o proxecto pretende combinar un enfoque práctico e tecnolóxico coa aplicación dos coñecementos adquiridos durante o ciclo formativo, desenvolvendo

unha solución web que responda a unha necesidade concreta dentro do ámbito da práctica musical.

2. Obxectivos

2.1. Obxectivo xeral

Deseñar e desenvolver unha aplicación web orientada á práctica musical que permita aos usuarios acceder a un catálogo de tablaturas, utilizar ferramentas de apoio á práctica e rexistrar o seu progreso mediante unha plataforma accesible e estruturada.

2.2. Obxectivos específicos

- Deseñar unha base de datos relacional que permita xestionar usuarios, cancións, tablaturas e o progreso de práctica dos usuarios garantindo a integridade da información.
- Implementar un sistema de autenticación e control de acceso que diferencie entre usuarios e administradores, permitindo a xestión e moderación do contido da plataforma.
- Desenvolver unha interface web responsive que facilite a navegación polo catálogo de tablaturas e a interacción cos distintos servizos da aplicación.
- Integrar ferramentas interactivas de apoio á práctica musical, como un metrónomo dixital, un afinador baseado en funcionalidades do navegador e un reprodutor de tablaturas con sincronización de son e visualización.

3. Fases de traballo

3.1. Fase de estudo

3.1.1. Plan de empresa

O plan de empresa que se presenta a continuación contextualiza o desenvolvemento de LumiMusic nun entorno profesional real, identificando a oportunidade de negocio e analizando a viabilidade técnica, económica e comercial do proxecto.

Descrición xeral da empresa

LumiMusic é unha empresa de base tecnolóxica especializada no desenvolvemento e explotación de plataformas web orientadas á práctica e formación musical. A empresa estará formada por unha única persoa, con perfil técnico en desenvolvemento de aplicacións web. O seu ramo de actividade é o sector tecnolóxico e de servizos educativos dixitais, con especial foco en usuarios que desexan aprender a tocar un instrumento ou practicar cancións de distintas nacionalidades e estilos musicais.

O contorno socioeconómico no que operará é principalmente dixital e global, dirixíndose a calquera usuario con acceso a internet e interese pola formación ou práctica musical, independentemente da súa localización. O obxectivo principal é ofrecer unha plataforma centralizada, accesible e intuitiva que mellore a experiencia de práctica musical dos seus usuarios. O horario de traballo será de luns a venres en xornada completa. O anagrama da empresa está formado polas iniciais "L" e "M" de LumiMusic, onde a "L" aparece no cor primario da marca (#84cc16) e a "M" no cor secundario (#4cd7f6).

Datos da empresa

Campo	Valor
Razón social	LumiMusic S.L.
Nome comercial	LumiMusic
Forma xurídica	Sociedade de Responsabilidade Limitada (S.L.)
Domicilio social	Galicia, España
Capital social	1000€
Sector de actividade	Desenvolvemento de software e servizos educativos dixitais (CNAE 6201)
Data prevista de inicio	Setembro de 2026
Numero de socios	1
Numero de traballadores previstos (1º ano)	1

O tipo de contrato laboral previsto será o réxime de autónomo para o socio fundador. A estrutura organizativa será unipersonal, asumindo o único empregado todas as funcións de desenvolvemento, mantemento e atención ao usuario. As obrigacións fiscais incluírán o IVE trimestral, o IRPF en declaracións trimestrais e a cota de autónomos mensual. En materia de prevención de riscos laborais, aplicarase o plan básico para traballadores autónomos, con especial atención aos riscos ergonómicos derivados do traballo con pantallas.

Datos económicos

Concepto	Importe
Fondos propios (capital inicial)	1000€
Fondos alleos	0€
Axudas e subvencións previstas	Bonificación cota autónomos (tarifa plana)
Investimento previsto (1º ano)	1000€
Previsión de ingresos (1º ano)	Variable segundo captación de usuarios
Previsión de gastos (1º ano)	~837 € (detallados no apartado de Plans)

Os gastos operativos do primeiro ano son reducidos grazas ao aproveitamento de licenzas de software existentes e á elección de infraestrutura de baixo custo. Prevese reinvestir os primeiros ingresos na mellora da plataforma e na ampliación do catálogo de contidos.

Datos de mercado

O público obxectivo de LumiMusic está formado por dous perfís principais: persoas que desexan aprender a tocar un instrumento desde cero e usuarios que xa tocan e queren practicar cancións de distintos estilos e nacionalidades, buscando un acceso organizado a tablaturas e recursos de apoio. Ambos perfís comparten o uso habitual de internet e a preferencia por ferramentas accesibles desde calquera dispositivo. A carta de servizos de LumiMusic inclúe: acceso a un catálogo de tablaturas organizado por instrumento, artista e nivel de dificultade, ferramentas interactivas de práctica (metrónomo e afinador), sistema de seguimento do progreso persoal e xestión de cursos estruturados por leccións. A estratexia de captación basearase en posicionamento orgánico en buscadores (SEO), presenza en redes sociais e comunidades de músicos en liña.

Procesos de produción

O produto principal é a plataforma web LumiMusic, desenvolvida mediante tecnoloxías web actuais (PHP con Laravel no backend, React no frontend, MySQL como sistema xestor de base de datos e Docker para a containerización). O proceso de produción inclúe as seguintes etapas: análise de requisitos, deseño da arquitectura e base de datos, desenvolvemento iterativo de funcionalidades, fase de probas e control de calidade, despregamento no servidor VPS e mantemento continuo da plataforma.

Os servizos ofertados contemplarán o acceso á plataforma en modalidade gratuíta con funcionalidades básicas e, nunha fase posterior, unha subscrición de pago con acceso completo ao catálogo e ferramentas avanzadas.

Recursos humanos e materiais

A empresa contará cun único traballador, o socio fundador, que asumirá os roles de desenvolvidor web full-stack, administrador do sistema e xestor de contido. En canto aos recursos materiais, o equipamento principal é un portátil de capacidades medias-altas (aprox. 800 €), que se empregará para o desenvolvemento, as probas e a xestión da plataforma. O software empregado inclúe: PHPStorm e Visual Studio Code como IDEs, DBBeaver para a administración da base de datos, Docker para a containerización da aplicación e Git con GitHub para o control de versións e aloxamento do repositorio. A licenza de PHPStorm non suporá un custo no primeiro ano ao dispoñerse xa dunha licenza vixente; prevese a súa renovación a partir do segundo ano. A infraestrutura de produción estará baseada nun VPS de IONOS cunha oferta inicial de 5 €/mes (IVE non incluído) durante os primeiros 6 meses, con 6 vCores de CPU, 8 GB de RAM e 240 GB de almacenamento NVMe, garantindo un rendemento axeitado para o volume de usuarios inicial. O dominio lumimusic.es estará rexistrado en Dondominio cun custo de 7 €/ano. Implementarase HTTPS en todas as comunicacións e unha política de copias de seguridade automatizadas mediante scripts de volcado da base de datos MySQL.

Creación e posta en marcha da empresa

A forma xurídica elixida é a Sociedade de Responsabilidade Limitada (S.L.), pola limitación da responsabilidade ao capital achegado. Os trámites administrativos previstos inclúen: certificación negativa de denominación social no Rexistro Mercantil, outorgamento de escritura pública de constitución ante notario, inscrición no Rexistro Mercantil, obtención do NIF e alta no IAE (epígrafe 879.1 - Servizos de información e informática). Xestionarase a solicitude da tarifa plana de autónomos para reducir os custos fixos durante os primeiros meses de actividade.

Plans

O investimento inicial é de 1.000 €, destinados principalmente á adquisición do equipamento de traballo (portátil, ~800 €) e aos gastos de constitución da empresa (~200 €).

Os gastos operativos anuais previstos son os seguintes:

Concepto	Custo
VPS IONOS (6 meses a 5 €/mes + 6 meses tarifa normal estimada)	~80€
Dominio lumimusic.es (Dondominio)	7€
Licenza PHPStorm (só a partir do 2º ano)	0€ (1º ano)
Total gastos operativos (1º ano)	~87€

O plan de prevención de riscos laborais contemplará a avaliación dos riscos ergonómicos e visuais derivados do traballo continuado con pantallas. A partir do segundo ano, prevese ampliar a infraestrutura se o crecemento da base de usuarios así o require, e incorporar a renovación da licenza de PHPStorm como gasto recorrente.

3.2. Planificación

Unha vez definido o proxecto a grandes liñas, establécese un plan de acción que permita levar a cabo o desenvolvemento da aplicación web de forma organizada e coherente. A planificación estrutúrase en función das fases principais e dos recursos necesarios, respondendo ás cuestións recomendadas para esta etapa.

Alcance do proxecto: O desenvolvemento centrarase nunha plataforma web orientada á práctica musical, incorporando funcionalidades como xestión de tablaturas, rexistro de progreso de usuarios, integración de ferramentas interactivas (metrónomo e afinador) e sincronización de son coa visualización de tablaturas.

Orzamento e recursos: O investimento inicial previsto é de 1.000 €, destinados principalmente á adquisición do equipamento de traballo (portátil, ~800 €) e aos gastos de constitución da empresa (~200 €). Os gastos operativos anuais son reducidos: o VPS de IONOS cunha oferta inicial de 5 €/mes durante os primeiros 6 meses (6 vCores, 8 GB RAM, 240 GB NVMe) e o dominio lumimusic.es rexistrado en Dondominio por 7 €/ano. A licenza de PHPStorm non supón custo no primeiro ano ao dispoñerse xa dunha licenza vixente. O custo operativo total estimado para o primeiro ano é de aproximadamente 97 €.

Recursos hardware e software: O equipamento principal consiste nun portátil de capacidades medias-altas. O stack tecnolóxico empregado inclúe PHP con Laravel e

React para o desenvolvemento da aplicación, MySQL como sistema xestor de base de datos, e Docker para a containerización do entorno. Como ferramentas de apoio utilízanse PHPStorm e Visual Studio Code como IDEs, DBeaver para a administración da base de datos, e Git con GitHub para o control de versións.

Riscos: Entre os riscos destacan a complexidade de integración das APIs de audio no navegador, posibles problemas de compatibilidade entre dispositivos e limitacións na carga de ficheiros por parte dos usuarios. Prevese mitigar estes riscos mediante probas iterativas e control de versións no repositorio.

Recursos humanos: O proxecto será desenvolvido por unha única persoa

Tarefas: As tarefas principais inclúen: análise e definición de requisitos, deseño da base de datos e da arquitectura, desenvolvemento de backend e frontend, integración das ferramentas interactivas, realización de probas unitarias e de integración, documentación e despregamento.

Fitos a cumprir: A planificación temporal do proxecto estrutúrase nas seguintes etapas orientativas:

Período	Fito
Mes 1	Finalización de análise de requisitos e deseño de base de datos
Mes 2-3	Desenvolvemento do backend e estrutura MVC con Laravel
Mes 3-4	Desenvolvemento do frontend con React e integración de ferramentas interactivas
Mes 5	Realización do plan de probas unitarias e de integración
Mes 6	Documentación técnica e despregamento da aplicación no VPS

3.3. Análise e deseño

Nesta fase analizarase detalladamente o proxecto, definindo os requirimentos funcionais e non funcionais necesarios para a súa correcta implementación. O obxectivo é garantir que a aplicación web cumpra coas necesidades dos usuarios,

sexa escalable e manteña un alto nivel de usabilidade e accesibilidade.

Análise de requirimentos:

Funcionais: O sistema debe permitir o rexistro e autenticación de usuarios, a xestión de tablaturas con revisión previa por parte do administrador, a integración de ferramentas interactivas como metrónomo e afinador, e o rexistro do progreso de práctica de cada usuario.

Non funcionais: A aplicación debe ser responsive, soportar múltiples dispositivos e navegadores, garantir a seguridade na autenticación e no almacenamento de datos, e ofrecer unha experiencia de usuario fluída sen retrasos perceptibles na interacción.

Deseño da arquitectura:

O proxecto seguirá unha arquitectura cliente-servidor baseada no patrón MVC en PHP. O backend xestionará a lóxica de negocio, a comunicación coa base de datos e o control de acceso por roles, mentres que o frontend proporcionará a interface interactiva, integrando JavaScript para a actualización dinámica de contidos e as ferramentas de apoio á práctica musical.

Deseño da interface:

A interface será responsiva e accesible, desenvolvida con React, HTML e CSS, co obxectivo de ofrecer unha experiencia intuitiva e agradable. As funcionalidades interactivas, como o metrónomo e o afinador, serán integradas de forma que non interrompan o fluxo de uso da aplicación e se adapten ao tamaño da pantalla e ao dispositivo do usuario.

Para a definición visual da interface, realizouse un prototipo da aplicación web que recolle as principais vistas e fluxos de navegación. Os mockups resultantes pódense consultar na carpeta Documentación/Mockups incluída na entrega do proxecto.

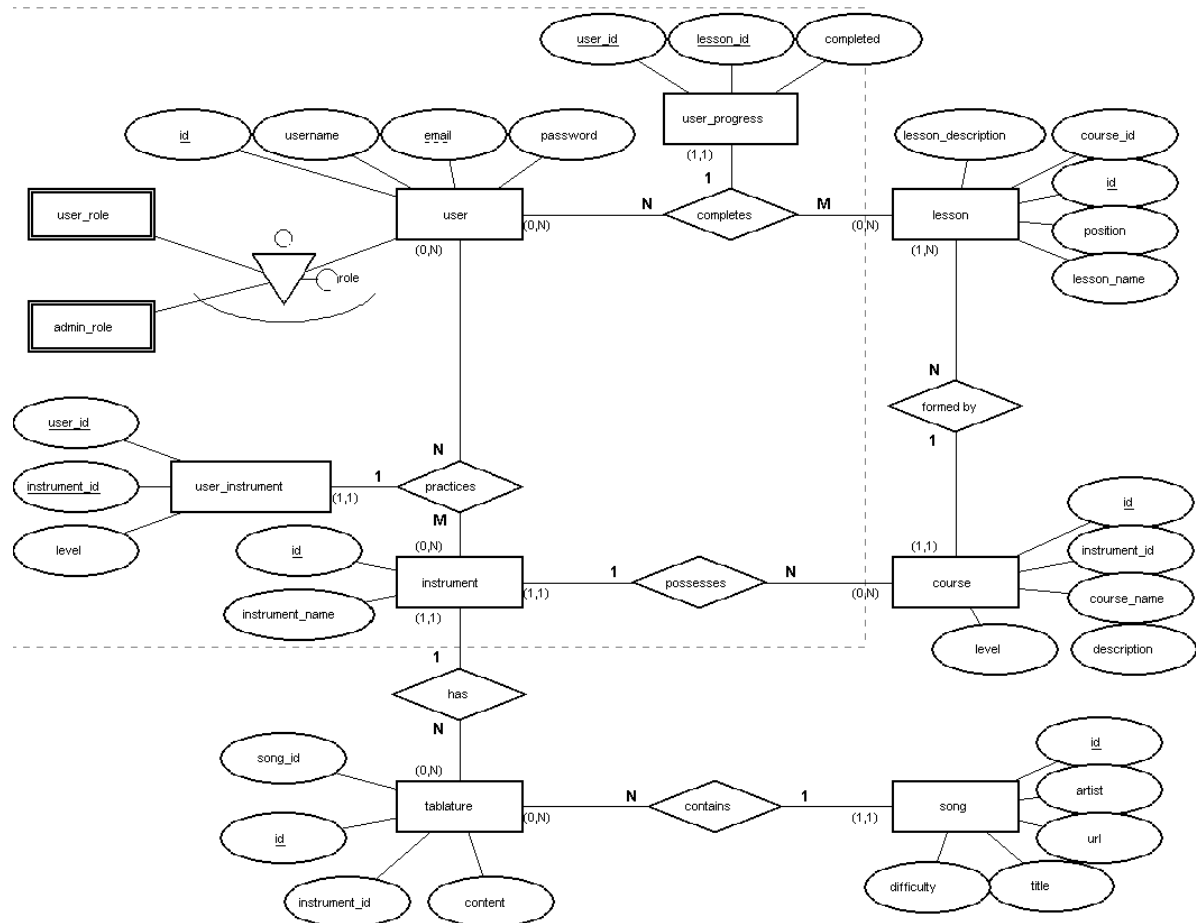
Consideracións de escalabilidade e seguridade:

O deseño contempla a posibilidade de escalar a aplicación en contedores Docker e implementar boas prácticas de seguridade, como cifrado de contrasinais e validación de entradas de usuario, garantindo un funcionamento robusto e seguro fronte a posibles vulnerabilidades.

Diseño da base de datos:

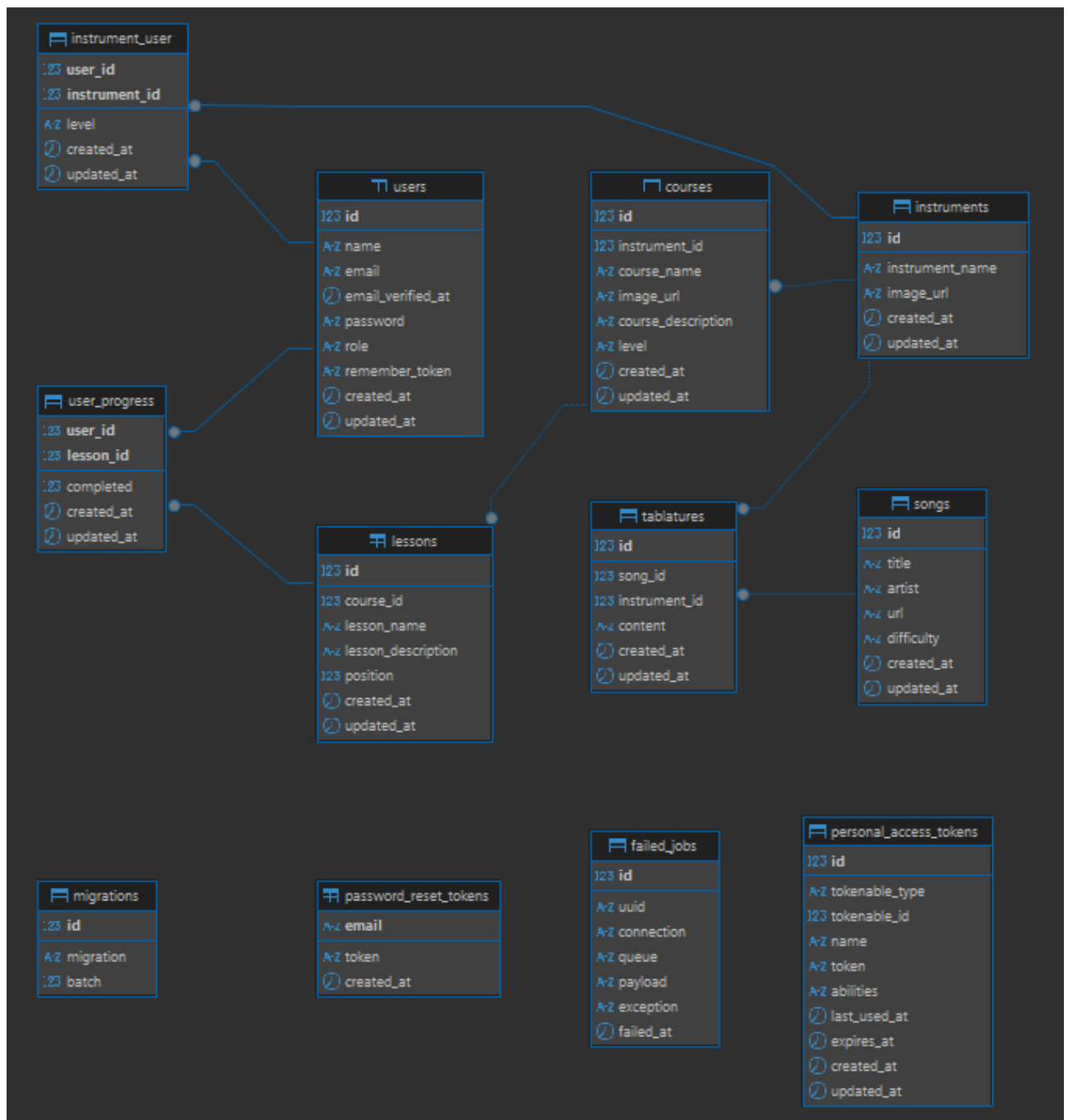
A base de datos relacional incluíra as seguintes entidades principais: Usuarios, Cancións, Tablaturas, Instrumentos, Cursos, Leccións e Roles. As relacións entre estas entidades garantirán a integridade referencial e permitirán consultas eficientes para xestionar o contido e o progreso dos usuarios. Os detalles dos tipos de datos pódense consultar no Anexo A

Diagrama entidade-relación:



O arquivo do deseño do modelo entidade-relación pódese tamén consultar na carpeta de Documentación, téñ o nome de: **diagrama-er-bbdd.dgrx**

Modelo relacional:



Implementación de especialización:

No diseño do modelo de datos, aplicouse un concepto de especialización na entidade USERS. Esta entidade actúa como unha clase xeral que se especializa en dous perfís distintos: 'admin' e 'user'. Aínda que no modelo relacional se implementou mediante un atributo role de tipo ENUM para optimizar o rendemento da base de datos, a nivel conceptual representa unha especialización total e disxunta, xa que cada rexistro da táboa debe pertencer obrigatoriamente a un destes dous tipos e non pode ser ambos á vez.

Modelo relacional:

USERS (id, name, email, password, role)

INSTRUMENTS (id, instrument_name, image_url)

SONGS (id, title, artist, url, difficulty)

TABLATURES (id, *instrument_id*, *song_id*, content)

COURSES (id, *instrument_id* course_name, course_description, level)

LESSONS (id, *course_id*, lesson_name, lesson_description, position)

USER_PROGRESS (user_id, lesson_id, completed)

USER_INSTRUMENT (user_id, instrument_id, level)

Nota: Os atributos entre asteriscos (*) fan referencia a Foreign Key

Relación Users-Instruments-Instrument_user: Relación N:M, un usuario pode estar asignado a varios instrumentos, e un instrumento pode estar asignado a varios usuarios; na táboa intermedia creada para almacenar estas relacións, hai un atributo para rexistrar o nivel de competencia do usuario co instrumento.

Relación Instruments-Courses: Relación 1:N, un instrumento pode ter moitos cursos pero un curso só pertence a un instrumento

Relación Courses-Lessons: Relación 1:N, un curso pode ter moitas leccións pero unha lección só pode pertencer a un curso

Relación Users-Lessons-User_progress: Relación N:M, un usuario pode estar asignado a varias leccións, e unha lección pode estar asignada a varios usuarios; a táboa intermedia contén un campo para rexistrar se o usuario completou a lección

Relación Instruments-Tablatures: Relación 1:N, un instrumento pode ter asignadas moitas tablaturas e unha tablatura só pode pertencer a un instrumento

Relación Songs-Tablatures: Relación 1:N, unha canción pode ter moitas tablaturas e unha tablatura só pode pertencer a unha canción

Consultas complexas clave para obter información:

1. Mostra qué cancións practicou cada usuario e en qué instrumento.

```
SELECT u.name, s.title as canción, i.instrument_name  
  
FROM users u  
  
JOIN instrument_user iu ON u.id = iu.user_id  
  
JOIN instruments i ON iu.instrument_id = i.id  
  
JOIN tablatures t ON i.id = t.instrument_id  
  
JOIN songs s ON t.song_id = s.id;
```

2. Recuento de lecciones completadas por curso

```
SELECT c.course_name, COUNT(up.lesson_id) as total_completadas  
  
FROM courses c  
  
JOIN lessons l ON c.id = l.course_id  
  
JOIN user_progress up ON l.id = up.lesson_id  
  
WHERE up.completed = 1  
  
GROUP BY c.id;
```

3. Cursos con algún estudiante

```
SELECT i.instrument_name, COUNT(iu.user_id) as estudiante  
  
FROM instruments i  
  
JOIN instrument_user iu ON i.id = iu.instrument_id  
  
GROUP BY i.id  
  
HAVING estudiante > 0 ;
```

4. Recupera nomes de canciones e cursos en nunha sola lista.

```
SELECT title as nome, 'Canción' as tipo
```

FROM songs s

UNION

SELECT course_name, 'Curso' FROM courses;

5. Usuarios que non completaron ninguna lección

SELECT u.name, u.email

FROM users u

LEFT JOIN user_progress up ON u.id = up.user_id

WHERE up.user_id IS NULL;

Debido á extensión e ao nivel de detalle técnico de cada entidade, o **Diccionario de Datos** completo, onde se especifican tipos de datos, restricións e descricións de cada campo, incluíuse no **Anexo A** deste documento para facilitar a fluidez da lectura

3.4. Desenvolvemento

Nesta fase descríbese a contorna técnica e as ferramentas empregadas para a construción do sistema, garantindo a súa escalabilidade e robustez, o xestor de base de datos usado para este proxecto é MySQL 8.0 (dockerizado), para o manexo do backend úsase PHP 8.1 co framework de Laravel e as migracions para o control de versións da base de datos, para o manexo do frontend usase React e para o manexo de versions do proxecto usase Git e Github

3.4.1. Entorno de desenvolvemento e ferramenta

Para a construción de LumiMusic, seleccionouse un conxunto de ferramentas de grao profesional que facilitan a análise, o deseño, a depuración e o despregamento da aplicación:

- **PHPStorm e Visual Studio Code:** Empregáronse ambos IDEs de forma complementaria. PHPStorm permitiu unha integración nativa co framework Laravel e soporte avanzado para a depuración, mentres que Visual Studio Code utilizouse pola súa versatilidade e lixeireza no desenvolvemento dos compoñentes de frontend.
- **DBeaver:** Esta ferramenta de administración de bases de datos utilizouse para a xestión, consulta e visualización do motor MySQL 8.0. Facilitou a execución de scripts e o mantemento da integridade referencial durante as fases de desenvolvemento.
- **DBDap:** O deseño inicial do esquema da base de datos e a creación do modelo entidade-relación realizouse mediante esta aplicación, o que permitiu definir as cardinalidades e relacións de forma visual antes da súa implementación física.
- **Docker:** Toda a infraestrutura, incluíndo o servidor web, a base de datos e o intérprete de PHP, foi dockerizada. Isto garante que a aplicación funcione de forma idéntica en calquera contorno, facilitando a súa posterior implantación.

3.4.2. Política de copias de seguridade

A política de copias de seguridade consiste nun volcado completo semanal da base de datos. Para iso, inclúese no proxecto o script backup.sh, situado no directorio raíz xunto ao arquivo docker-compose.yml, co seguinte contido:

```
#!/bin/bash
```

```
mkdir -p ./backups
```

```
docker exec mysql_db mysqldump -u root -proot lumimusic >  
./backups/backup_lumimusic_$(date +%Y%m%d).sql
```

En produción, este script executaríase de forma automatizada mediante un cron job no servidor VPS, programado para lanzarse cada domingo ás 2:00:

```
0 2 * * 0 /home/usuario/lumimusic/backup.sh
```

Isto garante que se xere semanalmente un volcado completo da base de datos na carpeta backups, identificado coa data de execución no nome do arquivo.

3.4.3. Programación Orientada a Obxectos (POO)

O desenvolvemento backend de LumiMusic aplica os principios fundamentais da Programación Orientada a Obxectos mediante o framework Laravel, construído sobre PHP e que fai un uso extensivo deste paradigma. A continuación descríbese como se aplica cada un dos conceptos principais no proxecto.

Herdanza

A herdanza permite que unha clase filla herde os atributos e métodos dunha clase pai, favorecendo a reutilización de código. En LumiMusic establécese unha cadea de herdanza en todos os controladores do proxecto: cada controlador concreto (AdminController, CourseController, AuthController, etc.) estende a clase Controller da aplicación, que á súa vez estende a clase base Illuminate\Routing\Controller proporcionada por Laravel.

```
class Controller extends BaseController
{
    use AuthorizesRequests, ValidatesRequests;
}
```

Controller.php

```
class AdminCourseController extends Controller
{
```

AdminCourseController.php

Grazas a esta cadea de herdanza, todos os controladores herdan de forma automática funcionalidades como a validación de peticións (ValidatesRequests) e o control de autorización (AuthorizesRequests), sen necesidade de reimplementar estes comportamentos en cada clase.

Instanciación

A instanciación é o proceso de creación de obxectos a partir dunha clase. En LumiMusic prodúcese principalmente a través dos modelos Eloquent de Laravel, tanto de forma explícita mediante asignación a variables, como de forma implícita mediante métodos estáticos.

```
$user = User::create([
    'name' => $validated['name'],
    'email' => $validated['email'],
    'password' => bcrypt($validated['password'])
]);
```

AuthController.php

Ademais, Laravel inxecta automaticamente instancias dos modelos nos métodos dos controladores mediante Route Model Binding, como ocorre en `CourseController::show(Course $course)`, onde o framework resolve e instancia o obxecto `Course` correspondente ao identificador da URL antes de executar o método.

Polimorfismo

O polimorfismo permite que distintas clases implementen métodos co mesmo nome pero con comportamentos diferentes. En LumiMusic aplícase mediante a sobreescritura de métodos en distintos controladores. Por exemplo, o método `destroy()` está presente en múltiples controladores, pero a súa implementación varía segundo o recurso que xestiona:

```
public function destroy($id)
{
    Course::destroy($id);
    return response()->json(['message' => 'Deleted'], status: 204);
}
```

AdminCourseController.php


```
public function destroy(Instrument $instrument)
{
    $instrument->delete();

    return response()->noContent();
}
```

InstrumentController.php

Do mesmo xeito, o método `show()` presenta comportamentos diferentes segundo o controlador: en `CourseController` carga as leccións ordenadas por posición e engade o estado de completado de cada unha para o usuario autenticado, mentres que en `SongController` simplemente devolve os datos da canción sen procesamento adicional.

Destrucción de obxectos e liberación de memoria

En PHP, a destrución de obxectos e a liberación de memoria xestiónanse de forma automática polo recolector de lixo da linguaxe, que libera a memoria ocupada polos obxectos cando estes deixan de ter referencias activas. No contexto de Laravel, isto ocorre ao finalizar cada ciclo de petición-resposta HTTP, momento no que todos os obxectos creados durante o procesamento son destruídos automaticamente. Non obstante, o proxecto inclúe casos de destrución explícita. Un exemplo é o peche de sesión en `AuthController`, onde o token de acceso se elimina de forma deliberada:

```
public function logout(Request $request)
{
    $request->user()->currentAccessToken()->delete();

    return response()->json(['message' => 'Logged out']);
}
```

AuthController.php

Neste caso, o token de autenticación xestionado por Laravel Sanctum é destruído

explicitamente na base de datos, invalidando o obxecto e liberando os recursos asociados á sesión do usuario.

3.4.4. Plan de probas

Nesta sección defínese o plan de probas unitarias e de integración realizado sobre o backend da aplicación LumiMusic, empregando PHPUnit, o framework de testing integrado en Laravel. O obxectivo é verificar que os principais módulos do sistema funcionan correctamente de forma illada e en conxunto. As probas execútanse sobre unha base de datos SQLite en memoria, illada da base de datos de produción, garantindo que non se altera ningún dato real durante o proceso de validación.

Probas unitarias

As probas unitarias verifican o comportamento de compoñentes illados do sistema, sen dependencias externas.

Id	Descricion	Ruta	Resultado esperado
U-01	Rexistro de usuario con datos válidos	POST /api/register	Usuario creado, resposta 200
U-02	Rexistro con email duplicado	POST /api/register	Erro de validación, resposta 422
U-03	Rexistro con contrasinal demasiado curto	POST /api/register	Erro de validación, resposta 422
U-04	Login con credenciais correctas	POST /api/login	Token xerado, resposta 200
U-05	Login con credenciais incorrecto	POST /api/login	Erro de autenticación, resposta 401
U-06	Creación de canción con datos válidos	POST /api/admin/songs	Canción creada, resposta 201
U-07	Creación de canción sen título	POST /api/admin/songs	Erro de validación, resposta 422
U-08	Obtención do listado de cancións	GET /api/songs	Lista de cancións, resposta 200
U-09	Eliminación de canción existente	DELETE /api/admin/songs/{id}	Canción eliminada, resposta 204

U-10	Obtención de canción inexistente	GET /api/songs/{id}	Resposta 404
------	-------------------------------------	---------------------	--------------

Probas de integración

As probas de integración verifican o comportamento do sistema cando varios compoñentes interactúan entre si, en particular o control de acceso por roles.

Id	Descrición	Rol	Resultado esperado
I-01	Usuario sen autenticar accede a ruta protexida	-	Redireccionamento, resposta 401
I-02	Usuario con rol 'user' intenta eliminar unha canción	user	Acceso denegado, resposta 403
I-03	Usuario con rol 'admin' elimina unha canción	admin	Canción eliminada, resposta 204
I-04	Usuario con rol 'user' accede ao seu progreso	user	Datos de progreso, resposta 200
I-05	Admin accede ao listado completo de usuarios	admin	Lista de usuarios, resposta 200
I-06	Usuario con rol 'user' intenta acceder ao listado de usuarios	user	Acceso denegado, resposta 403

```
PS C:\Users\Alejandro\Documents\GitHub\luminusic\Código\backend> php artisan test

PASS Tests\Unit\ExampleTest
✓ that true is true 0.33s

PASS Tests\Feature\AuthTest
✓ u01 registro con datos validos 7.82s
✓ u02 registro con email duplicado 2.53s
✓ u03 registro con contraseña corta 1.54s
✓ u04 login con credenciales correctas 1.71s
✓ u05 login con contraseña incorrecta 1.82s

PASS Tests\Feature\ExampleTest
✓ the application returns a successful response 1.18s

PASS Tests\Feature\RoleAccessTest
✓ i01 usuario sin autenticar en ruta protegida 1.68s
✓ i02 usuario regular no puede eliminar cancion 1.59s
✓ i03 admin puede eliminar cancion 1.53s
✓ i04 usuario normal accede a su progreso 1.58s
✓ i05 admin accede al listado de usuarios 1.61s
✓ i06 usuario normal no puede ver listado usuarios 1.62s

PASS Tests\Feature\SongTest
✓ u06 creacion cancion con datos validos 1.60s
✓ u07 creacion cancion sin titulo 1.55s
✓ u08 obtención listado canciones 1.57s
✓ u09 eliminación cancion existente 1.54s
✓ u10 obtención cancion inexistente 1.64s

Tests: 18 passed (31 assertions)
Duration: 37.70s
```

* Podese consultar a imaxe con maior resolución esta atópase na carpeta de Documentación co nome: **tests**

Informe de resultados

As probas executáronse mediante o comando php artisan test sobre o entorno de desenvolvemento local con base de datos SQLite en memoria. Os resultados obtidos son os seguintes:

Id	Resultado
U-01	Superada
U-02	Superada
U-03	Superada
U-04	Superada
U-05	Superada
U-06	Superada
U-07	Superada
U-08	Superada
U-09	Superada
U-10	Superada
I-01	Superada
I-02	Superada
I-03	Superada
I-04	Superada
I-05	Superada
I-06	Superada

3.4.5. Requisitos mínimos

Nesta sección descríbense os requisitos mínimos de hardware e software necesarios para despregar e executar a aplicación LumiMusic nun entorno de produción.

Requisitos de hardware

Compoñente	Mínimo recomendado
CPU	2 vCores
RAM	2 GB
Almacenamento	20 GB
Conexión a internet	Requerida

O servidor de produción empregado neste proxecto é un VPS de IONOS con 6 vCores, 8 GB de RAM e 240 GB de almacenamento NVMe, o cal supera con marxe os requisitos mínimos e garante un rendemento axeitado para o volume de usuarios inicial.

Para o despregamento da aplicación unicamente é necesario ter instalado Docker e Docker Compose, xa que toda a infraestrutura está containerizada.

Requisitos do cliente

Para acceder á aplicación non se require ningunha instalación adicional por parte do usuario. Unicamente é necesario dispor dun navegador web moderno con soporte para JavaScript activado, como Google Chrome, Mozilla Firefox, Safari ou Microsoft Edge, en calquera dispositivo de escritorio ou móbil.

Ademais, a aplicación inclúe PhpMyAdmin accesible en <http://localhost:8081>, que permite a inspección e xestión da base de datos desde o navegador sen necesidade de instalar ferramentas adicionais.

3.5. Documentación

A fase de documentación é fundamental no desenvolvemento de software, xa que facilita o mantemento da aplicación, proporciona unha visión global do proxecto e permite que calquera desenvolvedor poida comprender e traballar co código de forma eficiente. A documentación de LumiMusic estrutúrase en tres niveis: documentación interna do código, documentación externa da API REST e documentación arquitectónica mediante o diagrama de clases UML.

3.5.1. Documentación interna do código

A documentación interna realízase mediante comentarios PHPDoc en tódolos controladores Laravel do proxecto. Cada método está documentado coa súa descrición funcional, os parámetros de entrada anotados con `@param` e o valor de retorno indicado con `@return`. Este estándar de documentación é compatible con ferramentas de xeración automática de documentación como phpDocumentor.

3.5.2. Documentación da API REST

A API REST de LumiMusic permite a comunicación entre o frontend React e o backend Laravel. Tódolos endpoints, agás o rexistro e o login, requiren autenticación mediante token Bearer de Laravel Sanctum.

URL base: `http://localhost:8000/api`

Autenticación: `Authorization: Bearer {token}`

Autenticación

Método	Endpoint	Acceso	Descrición
POST	/register	Público	Rexistro de novo usuario
POST	/login	Público	Inicio de sesión
POST	/logout	Usuario	Peche de sesión
PUT	/user/password	Usuario	Cambio de contrasinal

Cancións

Método	Endpoint	Acceso	Descrición
GET	/songs	Usuario	Listado de cancións
GET	/songs/{id}	Usuario	Datos dunha canción
GET	/songs/{id}/tablatures	Usuario	Tablaturas dunha can-

			ción
POST	/admin/songs	Admin	Crear canción
PUT	/admin/songs/{id}	Admin	Actualizar canción
DELETE	/admin/songs/{id}	Admin	Eliminar canción

Instrumentos

Método	Endpoint	Acceso	Descripción
GET	/instruments	Usuario	Listado de instrumentos
GET	/instruments/{id}	Usuario	Cursos dun instrumento
GET	/instruments/{id}/courses	Usuario	Cursos con detalle
POST	/admin/instruments	Admin	Crear instrumento
PUT	/admin/instruments/{id}	Admin	Actualizar instrumento
DELETE	/admin/instruments/{id}	Admin	Eliminar instrumento

Cursos

Método	Endpoint	Acceso	Descripción
GET	/courses	Usuario	Listado de cursos
GET	/courses/{id}	Usuario	Detalle dun curso con leccións e progreso
GET	/my-courses	Usuario	Cursos dos instrumentos do usuario
POST	/admin/courses	Admin	Crear curso
PUT	/admin/courses/{id}	Admin	Actualizar curso
DELETE	/admin/courses/{id}	Admin	Eliminar curso

Leccións

Método	Endpoint	Acceso	Descripción
GET	/lessons	Usuario	Listado de leccións
GET	/lessons/{id}	Usuario	Detalle dunha lección
POST	/admin/lessons	Admin	Crear lección
PUT	/admin/lessons/{id}	Admin	Actualizar lección
DELETE	/admin/lessons/{id}	Admin	Eliminar lección

Tablaturas

Método	Endpoint	Acceso	Descripción
GET	/tablatures	Usuario	Listado de tablaturas
GET	/tablatures/{id}	Usuario	Detalle dunha tablatura
POST	/tablatures	Usuario	Crear tablatura

PUT	/tablatures/{id}	Usuario	Actualizar tablatura
DELETE	/tablatures/{id}	Usuario	Eliminar tablatura

Progreso do usuario

Método	Endpoint	Acceso	Descripción
GET	/progress	Usuario	Progreso do usuario autenticado
GET	/progress/{id}	Usuario	Detalle dun rexistro de progreso
POST	/progress	Usuario	Marcar lección como completada
PUT	/progress/{id}	Usuario	Actualizar progreso
DELETE	/progress/{id}	Usuario	Eliminar rexistro de progreso

Instrumentos do usuario

Método	Endpoint	Acceso	Descripción
GET	/user-instruments	Usuario	Instrumentos do usuario autenticado
POST	/user-instruments	Usuario	Asociar instrumento ao usuario
DELETE	/user-instruments/{id}	Usuario	Desasociar instrumento

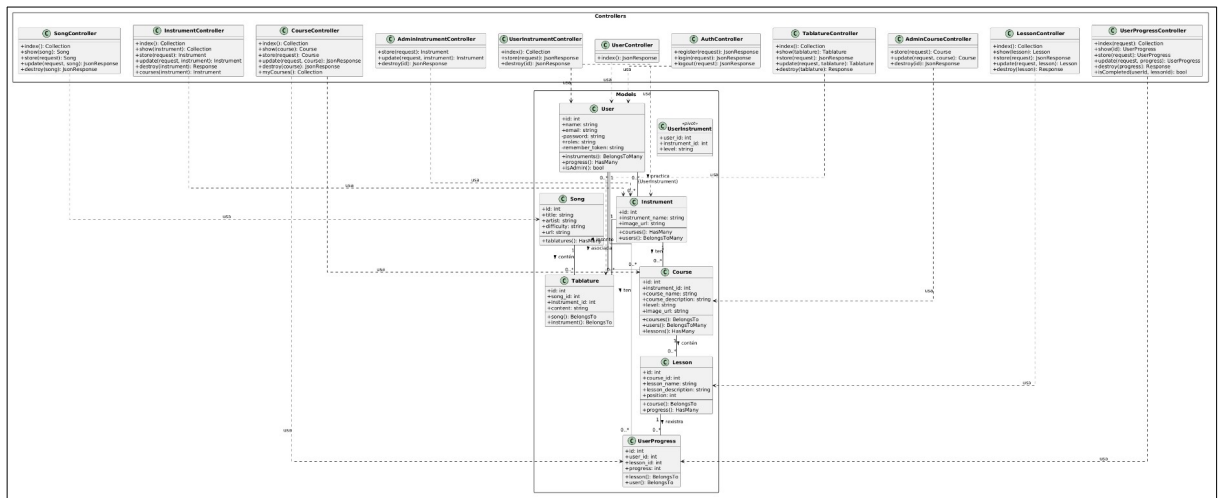
Administración de usuarios

Método	Endpoint	Acceso	Descripción
GET	/admin/users	Admin	Listado completo de usuarios
PUT	/admin/users/{id}	Admin	Actualiza os datos dun usuario
DELETE	/admin/users/{id}	Admin	Borra un usuario

3.5.3. Diagrama de clases UML

O diagrama de clases UML representa a arquitectura orientada a obxectos da aplicación LumiMusic, mostrando as clases principais do sistema, os seus atributos e métodos, e as relacións entre elas. O diagrama está organizado en dous paquetes: **Models**, que contén as clases Eloquent con as súas relacións, e **Controllers**, que contén os controladores da API REST con os seus métodos públicos. As liñas sólidas representan relacións entre modelos e as liñas punteadas representan

dependencias dos controladores cara aos modelos. Este esquema foi feito coa ferramenta web PlantUML



* Por se se quere consultar a imaxe con maior resolución esta atópase na carpeta de Documentación co nome: esquema-uml-lumimusic

4. Conclusións

O obxectivo xeral deste proxecto era deseñar e desenvolver unha aplicación web orientada á práctica musical que permitise aos usuarios acceder a un catálogo de tablaturas, utilizar ferramentas de apoio á práctica e rexistrar o seu progreso mediante unha plataforma accesible e estruturada. Tras o proceso de desenvolvemento, pódese concluír que o proxecto acadou parcialmente os obxectivos formulados inicialmente. Os aspectos relacionados coa estrutura da aplicación, a xestión da información, o control de acceso e a interface de usuario foron implementados satisfactoriamente. Non obstante, algunhas das ferramentas de apoio á práctica musical previstas non chegaron a completarse dentro do prazo dispoñible, polo que determinados obxectivos quedaron acadados só de forma parcial.

O primeiro obxectivo específico, o deseño dunha base de datos relacional que xestionase usuarios, cancións, tablaturas e o progreso de práctica dos usuarios garantindo a integridade da información, cumpriuse de forma completa. O modelo relacional resultante conta con oito entidades correctamente relacionadas, con claves primarias e foráneas definidas, cardinalidades especificadas e unha especialización implementada na entidade de usuarios mediante un atributo de rol. O segundo obxectivo, a implementación dun sistema de autenticación e control de acceso que diferenciase entre usuarios e administradores, tamén se cumpriu satisfactoriamente. A integración de Laravel Sanctum permitiu xestionar a autenticación mediante tokens e implementar un middleware de control de acceso por roles, garantindo que as operacións de administración queden restrinxidas ao perfil correspondente.

O terceiro obxectivo, o desenvolvemento dunha interface web responsive, acadouse de forma satisfactoria. A aplicación frontend desenvolta con React adapta o seu deseño aos distintos tamaños de pantalla, ofrecendo unha experiencia de navegación coherente tanto en dispositivos de escritorio como en dispositivos móbiles.

O cuarto obxectivo, a integración de ferramentas interactivas de apoio á práctica musical, cumpriuse de forma parcial. O afinador foi implementado, aínda que o seu funcionamento non é completamente estable. O metrónomo e o reprodutor de tablaturas, que formarían parte desta funcionalidade, non chegaron a implementarse dentro do prazo do proxecto, quedando como liñas de traballo futuro.

Como principal achega, este proxecto demostra a viabilidade de desenvolver unha plataforma web funcional de práctica musical empregando tecnoloxías actuais do sector, integrando de forma coherente un backend API REST con Laravel, un frontend dinámico con React e unha base de datos relacional MySQL, todo nun entorno containerizado con Docker. O resultado obtido constitúe unha base sólida e escalable sobre a que continuar desenvolvendo funcionalidades, demostrando que os coñecementos adquiridos ao longo do ciclo formativo son suficientes para abordar proxectos web de complexidade real.

5. Propostas de mellora e valoración

Como melloras a futuro propónse mellorar o funcionamento do afinador, implementar o metrónomo e o reprodutor de tablaturas para que a aplicación poida ofrecer unha experiencia de práctica musical máis completa.

Durante o desenvolvemento atopáronse diferentes dificultades técnicas que permitiron adquirir experiencia práctica. Un dos casos máis representativos foi o afinador. Nunha primeira versión esixíase que a frecuencia detectada coincidise practicamente de forma exacta coa frecuencia teórica da nota, o que facía moi difícil que o usuario obtivese unha afinación correcta debido ás pequenas variacións naturais do son captado polo micrófono. Tras realizar varias probas, foi necesario ampliar a marxe de tolerancia para conseguir un funcionamento máis realista e utilizable.

Outro aspecto que supuxo unha curva de aprendizaxe foi o desenvolvemento do frontend con React. Inicialmente reiniciábase manualmente o servidor de desenvolvemento para comprobar os cambios realizados na interface. Máis adiante descubriuse o sistema de actualización automática incluído por defecto nas ferramentas de desenvolvemento, o que permitiu axilizar considerablemente o proceso de programación e probas

En canto á valoración xeral do proxecto, considero que o desenvolvemento de LumiMusic permitíume aplicar de forma práctica gran parte dos coñecementos adquiridos durante o ciclo. Ademais de traballar con bases de datos, APIs REST, Laravel, React e Docker, tamén serviu para aprender a resolver problemas reais no ámbito de desenvolvemento. Aínda que quedaron funcionalidades pendentes de implementar, o resultado final constitúe unha base sólida sobre a que continuar ampliando o proxecto no futuro.

6. Bibliografía

Laravel LLC. (2024). *Laravel 11.x Documentation*. <https://laravel.com/docs/11.x>

Meta Open Source. (2024). *React: The library for web and native user interfaces*. <https://react.dev>

Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>

Docker Inc. (2024). *Docker Documentation*. <https://docs.docker.com>

Sebastian Bergmann. (2024). *PHPUnit Documentation*. <https://docs.phpunit.de>

Anexo A. Diccionario de datos

User				
Campo	Tipo	Nulo	Atributos	Descripción
Id	Bigint	Non	PK, Unsigned	Identificador único do usuario
Name	Varchar (255)	Non		Nome do usuario
Email_verified_at	Varchar (255)	Si		Data da verificación do email
Password	Varchar (255)	Non		Contraseñal do usuario
Role	Enum	Non		Rol do usuario
Remember_token	Varchar (100)	Non		Campo de seguridade de laravel
Created_at	Timestamp	Non		Data da creación do usuario
Updated_at	Timestamp	Non		Data de actualización do usuario

* Role pode tomar os valores 'admin' o 'user'

Instruments				
Campo	Tipo	Nulo	Atributos	Descripción
Id	Bigint	Non	PK, Unsigned	Identificador único do instrumento
Instrument_name	Varchar (255)	Non		Nome do instrumento
Image_url	Text	Si		Url á imaxe do instrumento
Created_at	Timestamp	Non		Data de creación do instrumento
Updated_at	Timestamp	Non		Data de actualización do instrumento

Courses				
Campo	Tipo	Nulo	Atributos	Descrición
Id	Bigint	Non	PK, Unsigned	Identificador único do curso
Instrument_id	Bigint	Non	FK, Unsigned	Identificador único do instrumento
Course_name	Varchar (255)	Non		Nome do curso
Image_url	Text	Si		Url da imaxe do curso
Course_description	Text	Si		Descrición do curso
Level	Enum	Si		Nivel de dificultade do curso
Created_at	Timestamp	Non		Data da creación do curso
Updated_at	Timestamp	Non		Data da última actualización do curso

* Level pode tomar os valores 'Pincipiante', 'Intermedio' ou 'Avanzado'

Lessons				
Campo	Tipo	Nulo	Atributos	Descrición
Id	Bigint	Non	PK, Unsigned	Identificador único da lección
Course_id	Bigint	Non	FK, Unsigned	Identificador único do curso
Lesson_name	Varchar (255)	Non		Nome da lección
Lesson_description	Text	Si		Descrición da lección
Position	Int	Si		Posición da lección dentro do curso
Created_at	Timestamp	Non		Data da creación da lección
Updated_at	Timestamp	Non		Data da última actualización da lección

Instrument_user				
Campo	Tipo	Nulo	Atributos	Descrición
User_id	Bigint	Non	PK, FK, Unsigned	Identificador único do usuario
Instrument_id	Bigint	Non	PK, FK, Unsigned	Identificador único do instrumento
Level	Enum	Non		Nivel do usuario en relación ao instrumento
Created_at	Timestamp	Non		Data da creación da relación
Updated_at	Timestamp	Non		Data de actualización da relación

* Level pode tomar os valores 'Pincipiante', 'Intermedio' ou 'Avanzado'

User_progress				
Campo	Tipo	Nulo	Atributos	Descrición
User_id	Bigint	Non	PK, FK, Unsigned	Identificador único do usuario
Lesson_id	Bigint	Non	PK, FK, Unsigned	Identificador único da lección
Completed	Tinyint (1)	Non		Atributo para saber se a lección foi completada
Created_at	Timestamp	Non		Data da creación da lección
Updated_at	Timestamp	Non		Data de actualización da lección

Tablatures				
Campo	Tipo	Nulo	Atributos	Descrición
Id	Bigint	Non	PK, Unsigned	Identificador único da tablatura
Song_id	Bigint	Non	FK, Unsigned	Identificador único da canción
Instrument_id	Bigint	Non	FK, Unsigned	Identificador único do instrumento
Content	Longtext	Si		Contido da tablatura
Created_at	Timestamp	Non		Data de creación da tablatura
Updated_at	Timestamp	Non		Data de actualización da tablatura

Songs				
Campo	Tipo	Nulo	Atributos	Descripción
Id	Bigint	Non	PK, Unsigned	Identificador único da canción
Title	Varchar (255)	Non		Título da canción
Artist	Varchar (255)	Non		Artista/s da canción
Url	Text	Si		Url á canción
Difficulty	Varchar (255)	Si		Dificultade da canción
Created_at	Timestamp	Non		Data de creación da canción
Updated_at	Timestamp	Non		Data de actualización da canción

*Difficulty pode tomar os valores 'Principiante', 'Intermedio' ou 'Avanzado'